E-commerce API - MAX 4: Security

Gulzira Baiyeva -70259

Data: 09.05.26

The API handles errors using the following HTTP status codes: 403 if there is no token, 401 if the token is invalid. Data is validated using @Valid. The business logic is implemented in the OrderService and UserService; the controllers only handle requests. Access to /orders requires login.

*I ran from Maven and opened it while running the test, and it's working that way.

At the end I was need to get new "Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJhZG1pbiIsImlhdCI6MTc3ODM1MDIzMiwiZXhwIjoxNzc4NDM2NjMyfQ." for (# 3. Login - Should return a long token = Status 200
POST http://localhost:8080/auth/login
Content-Type: application/Json)

And then I got a new Authorization bearer, replaced in #5 to get endpoint

EcommerceApiApplication.java

```
package com.example.ecommerceapi;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication  // ← No exclude anymore
public class EcommerceApiApplication {
    public static void main(String[] args) {
        SpringApplication.run(EcommerceApiApplication.class, args);
    }
}
```

Controller (AuthController)

```
package com.example.ecommerceapi.controller;

import com.example.ecommerceapi.model.User;
import com.example.ecommerceapi.repository.UserRepository;
import com.example.ecommerceapi.security.JwtUtil;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.web.bind.annotation.*;
import java.util.Map;

@RestController
@RequestMapping("/auth")
public class AuthController {

    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;
    private final JwtUtil jwtUtil;

    public AuthController(UserRepository userRepository, PasswordEncoder
passwordEncoder, JwtUtil jwtUtil) {
        this.userRepository = userRepository;
        this.passwordEncoder = passwordEncoder;
        this.jwtUtil = jwtUtil;
    }

    @PostMapping("/register")
    public String register(@RequestBody User user) {
        user.setPassword(passwordEncoder.encode(user.getPassword())); //
Encrypt password
        userRepository.save(user);
        return "User registered";
    }

    @PostMapping("/login")
    public Map<String, String> login(@RequestBody User user) {
        User dbUser = userRepository.findByUsername(user.getUsername())
                .orElseThrow(() -> new RuntimeException("User not found"));

        if (passwordEncoder.matches(user.getPassword(),
```

```
dbUser.getPassword())) {
            String token = jwtUtil.generateToken(user.getUsername());
            return Map.of("token", token);
        }
        throw new RuntimeException("Invalid password");
    }
}
```

OrderController

```
package com.example.ecommerceapi.controller;

import com.example.ecommerceapi.model.Order;
import com.example.ecommerceapi.repository.OrderRepository;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
public class OrderController {

    private final OrderRepository orderRepository;

    public OrderController(OrderRepository orderRepository) {
        this.orderRepository = orderRepository;
    }

    @GetMapping("/orders")
    public List<Order> getOrders() {
        return orderRepository.findAll();
    }

    @PostMapping("/orders")
    public Order createOrder(@RequestBody Order order) {
        return orderRepository.save(order);
    }
}
```

ProductController

```
package com.example.ecommerceapi.controller;

import com.example.ecommerceapi.model.Product;
import com.example.ecommerceapi.repository.ProductRepository;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
public class ProductController {
    private final ProductRepository productRepository;

    public ProductController(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }
    @GetMapping("/products")
    public List<Product> getProducts() {
        return productRepository.findAll();
    }

    @PostMapping("/products")
    public Product createProduct(@RequestBody Product product) {
        return productRepository.save(product);
    }
}
```

Model (Order)

```
package com.example.ecommerceapi.model;

import jakarta.persistence.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "orders") // "order" is a SQL keyword
public class Order {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
```

```
    private String productName;
    private Integer quantity;
    private LocalDateTime orderDate = LocalDateTime.now();

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getProductName() { return productName; }
    public void setProductName(String productName) { this.productName =
productName; }
    public Integer getQuantity() { return quantity; }
    public void setQuantity(Integer quantity) { this.quantity = quantity; }
    public LocalDateTime getOrderDate() { return orderDate; }
    public void setOrderDate(LocalDateTime orderDate) { this.orderDate =
orderDate; }
}
```

Model(Product)

```
package com.example.ecommerceapi.model;

import jakarta.persistence.*;

@Entity
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private Double price;

    // Getters and setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public Double getPrice() { return price; }
    public void setPrice(Double price) { this.price = price; }
}
```

Model(User)

```
package com.example.ecommerceapi.model;

import jakarta.persistence.*;

@Entity
@Table(name = "users") // "user" is reserved in SQL
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true)
    private String username;
    private String password; // We'll store encrypted

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getUsername() { return username; }
    public void setUsername(String username) { this.username =
username; }
    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }
}
```

Repository (OrderRepository)

```
package com.example.ecommerceapi.repository;

import com.example.ecommerceapi.model.Order;
import org.springframework.data.jpa.repository.JpaRepository;

public interface OrderRepository extends JpaRepository<Order, Long> {
}
```

Repository(ProductRepository)

```
package com.example.ecommerceapi.repository;

import com.example.ecommerceapi.model.Product;
```

```
import org.springframework.data.jpa.repository.JpaRepository;

public interface ProductRepository extends JpaRepository<Product, Long> {
}
```

Repository(UserRepository)

```
package com.example.ecommerceapi.repository;

import com.example.ecommerceapi.model.User;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;

public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByUsername(String username);
}
```

Security(CustomerUserDetailsService)

```
package com.example.ecommerceapi.security;

import com.example.ecommerceapi.model.User;
import com.example.ecommerceapi.repository.UserRepository;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import
org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;

import java.util.ArrayList;

@Service
public class CustomUserDetailsService implements UserDetailsService {

    private final UserRepository userRepository;

    public CustomUserDetailsService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }
```

```
    @Override
    public UserDetails loadUserByUsername(String username) throws
UsernameNotFoundException {
        User user = userRepository.findByUsername(username)
                .orElseThrow(() -> new UsernameNotFoundException("User not
found"));
        return new
org.springframework.security.core.userdetails.User(user.getUsername(),
user.getPassword(), new ArrayList<>());
    }
}
```

Security(JwtAuthenticationFilter)

```
package com.example.ecommerceapi.security;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import
org.springframework.security.authentication.UsernamePasswordAuthenticati
onToken;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import
org.springframework.security.web.authentication.WebAuthenticationDetailsS
ource;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;

@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtUtil jwtUtil;
    private final UserDetailsService userDetailsService;
```

```
    public JwtAuthenticationFilter(JwtUtil jwtUtil, UserDetailsService
userDetailsService) {
        this.jwtUtil = jwtUtil;
        this.userDetailsService = userDetailsService;
    }

    @Override
    protected void doFilterInternal(HttpServletRequest request,
HttpServletResponse response, FilterChain chain)
            throws ServletException, IOException {

        String authHeader = request.getHeader("Authorization");
        String token = null;
        String username = null;

        if (authHeader != null && authHeader.startsWith("Bearer ")) {
            token = authHeader.substring(7);
            username = jwtUtil.extractUsername(token);
        }

        if (username != null &&
SecurityContextHolder.getContext().getAuthentication() == null) {
            UserDetails userDetails =
userDetailsService.loadUserByUsername(username);
            if (jwtUtil.validateToken(token, userDetails)) {
                UsernamePasswordAuthenticationToken authToken = new
UsernamePasswordAuthenticationToken(
                        userDetails, null, userDetails.getAuthorities());
                authToken.setDetails(new
WebAuthenticationDetailsSource().buildDetails(request));

SecurityContextHolder.getContext().setAuthentication(authToken);
            }
        }
        chain.doFilter(request, response);
    }
}
```

Security (JwtUtil)

```
package com.example.ecommerceapi.security;

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.SignatureAlgorithm;
import io.jsonwebtoken.security.Keys;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.stereotype.Component;

import java.security.Key;
import java.util.Date;
import java.util.function.Function;

@Component
public class JwtUtil {

    // Key term: Secret key = password used to sign the JWT. Keep it long.
    private final String SECRET =
"mySecretKeyForJwtTokenGenerationThatShouldBeVeryLongAndSecure1234
5";
    private final Key key = Keys.hmacShaKeyFor(SECRET.getBytes());

    public String generateToken(String username) {
        return Jwts.builder()
                .setSubject(username)
                .setIssuedAt(new Date())
                .setExpiration(new Date(System.currentTimeMillis() + 1000 * 60
* 60 * 24)) // 24 hours
                .signWith(key, SignatureAlgorithm.HS256)
                .compact();
    }

    public String extractUsername(String token) {
        return extractClaim(token, Claims::getSubject);
    }
```

```
    public boolean validateToken(String token, UserDetails userDetails) {
        final String username = extractUsername(token);
        return username.equals(userDetails.getUsername())
&& !isTokenExpired(token);
    }

    private Boolean isTokenExpired(String token) {
        return extractExpiration(token).before(new Date());
    }

    private Date extractExpiration(String token) {
        return extractClaim(token, Claims::getExpiration);
    }

    private <T> T extractClaim(String token, Function<Claims, T>
claimsResolver) {
        final Claims claims =
Jwts.parserBuilder().setSigningKey(key).build().parseClaimsJws(token).getB
ody();
        return claimsResolver.apply(claims);
    }
}
```

Security(SecurityConfig)

```
package com.example.ecommerceapi.security;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import
org.springframework.security.config.annotation.web.builders.HttpSecurity;
import
org.springframework.security.config.annotation.web.configuration.EnableWe
bSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
```

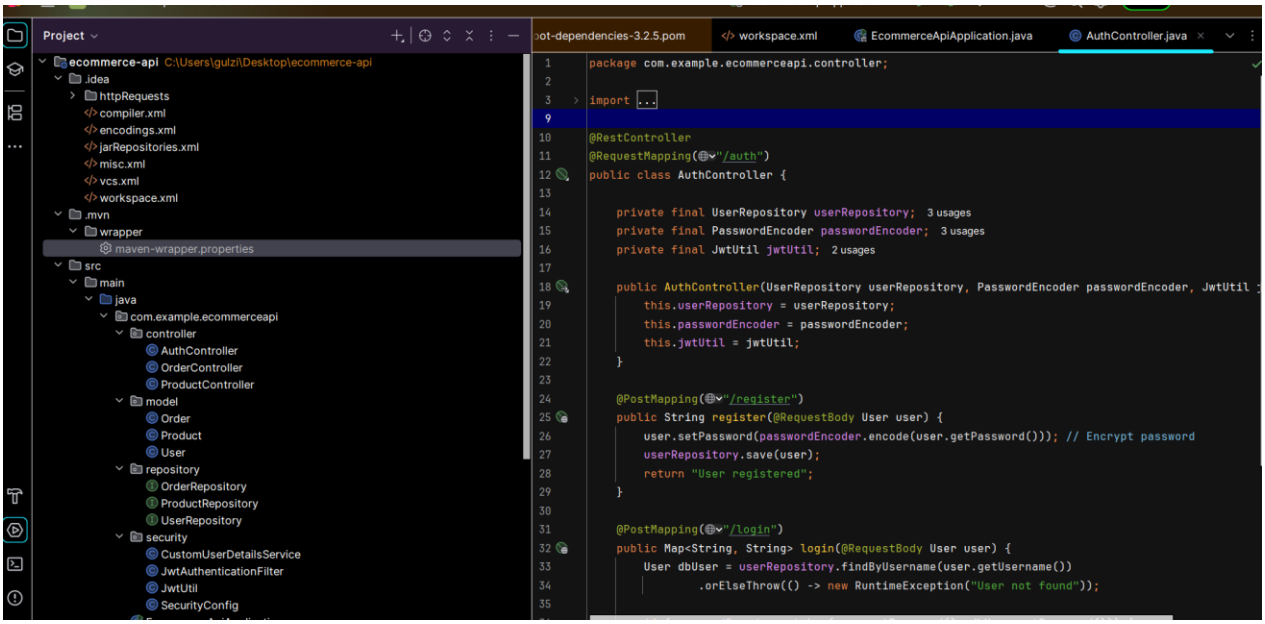
```
import
org.springframework.security.web.authentication.UsernamePasswordAuthent
icationFilter;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    private final JwtAuthenticationFilter jwtAuthenticationFilter;

    public SecurityConfig(JwtAuthenticationFilter jwtAuthenticationFilter) {
        this.jwtAuthenticationFilter = jwtAuthenticationFilter;
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception
{
        http.csrf(csrf -> csrf.disable())
                .authorizeHttpRequests(auth -> auth
                        .requestMatchers("/auth/**", "/h2-console/**").permitAll()
                        .requestMatchers(HttpMethod.GET, "/products").permitAll()
                        .anyRequest().authenticated()
                )
                .headers(headers -> headers.frameOptions(frameOptions ->
frameOptions.disable()))
                .sessionManagement(sess ->
sess.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
                .addFilterBefore(jwtAuthenticationFilter,
UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }
}
```

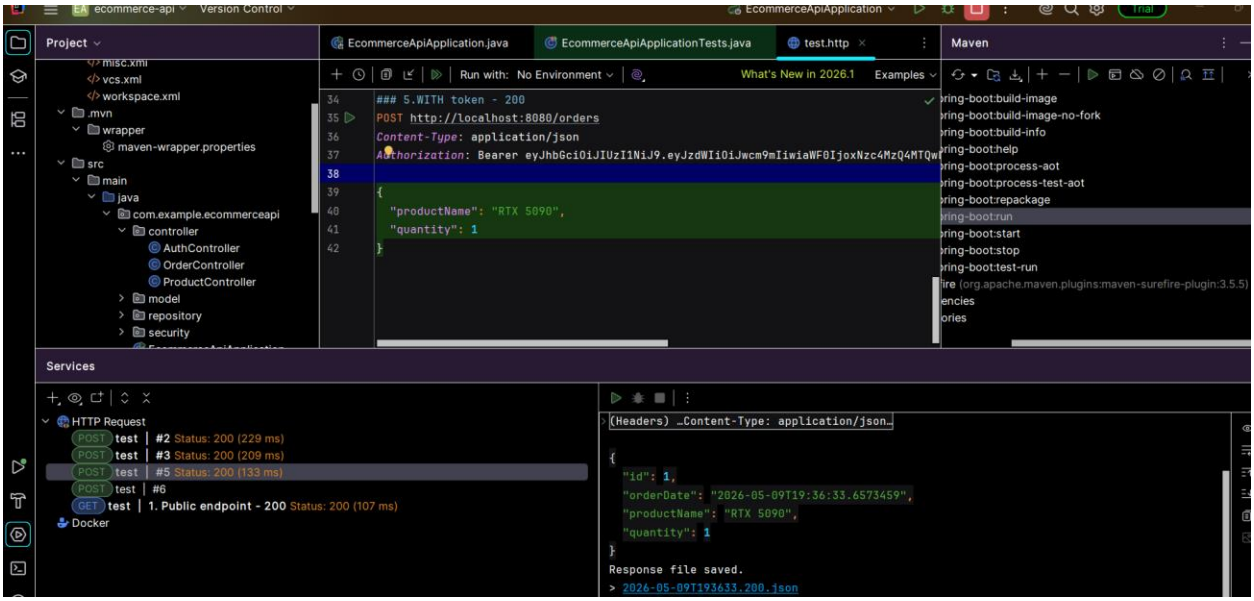
Test(test.http)

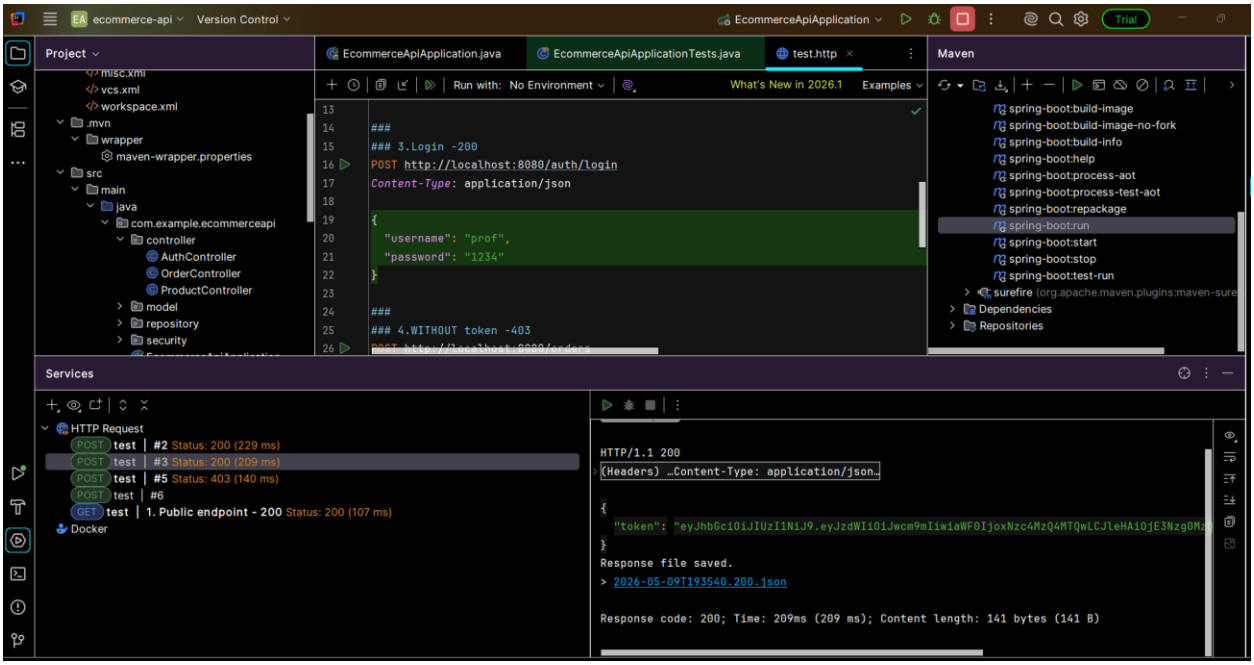
```
### 1. Public endpoint - 200
GET http://localhost:8080/products

###
### 2. Register - 200  + "User registered"
POST http://localhost:8080/auth/register
Content-Type: application/json

{
  "username": "prof",
  "password": "1234"
}

###
### 3.Login -200
POST http://localhost:8080/auth/login
Content-Type: application/json

{
  "username": "prof",
  "password": "1234"
}

###
### 4.WITHOUT token -403
POST http://localhost:8080/orders
Content-Type: application/json

{
  "productName": "RTX 5090",
  "quantity": 1
}
###
### 5.WITH token - 200
POST http://localhost:8080/orders
Content-Type: application/json
```

Authorization: Bearer
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJwcm9mIiwiaWF0IjoxNzc4MjI0Nzc5LCJleHA
iOjE3NzgzMTExNzl9.HFkCahEcHkJhoz-MPxZW5K7Ks_46C6Nl6c0co-u8VCk


```
{
  "productName": "RTX 5090",
  "quantity": 1
}
```


ecommerce-api
Version Control
EcommerceApiApplication
Trial
Project
EcommerceApiApplication.java
EcommerceApiApplicationTests.java
test.http
Maven
Run with: No Environment
What's New in 2026.1
Examples
misc.xml
vcs.xml
workspace.xml
.mvn
wrapper
maven-wrapper.properties
src
main
java
com.example.ecommerceapi
controller
AuthController
OrderController
ProductController
model
repository
security
5.WITH token - 200
POST http://localhost:8080/orders
Content-Type: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJwcm9mIiwiaWF0IjoxNzc4MzQ4MTQw
{
"productName": "RTX 5090",
"quantity": 1
}
spring-boot:build-image
spring-boot:build-image-no-fork
spring-boot:build-info
spring-boot:help
spring-boot:process-aot
spring-boot:process-test-aot
spring-boot:repackage
spring-boot:run
spring-boot:start
spring-boot:stop
spring-boot:test-run
Services
HTTP Request
POST test | #2 Status: 200 (229 ms)
POST test | #3 Status: 200 (209 ms)
POST test | #5 Status: 200 (133 ms)
POST test | #6
GET test | 1. Public endpoint - 200 Status: 200 (107 ms)
Docker
(Headers) …Content-Type: application/json…
{
"id": 1,
"orderDate": "2026-05-09T19:36:33.6573459",
"productName": "RTX 5090",
"quantity": 1
}
Response file saved.
> 2026-05-09T193633.200.json


ecommerce-api
Version Control
EcommerceApiApplication
Trial
Project
EcommerceApiApplication.java
EcommerceApiApplicationTests.java
test.http
Maven
Run with: No Environment
What's New in 2026.1
Examples
misc.xml
vcs.xml
workspace.xml
.mvn
wrapper
maven-wrapper.properties
src
main
java
com.example.ecommerceapi
controller
AuthController
OrderController
ProductController
model
repository
security
###
3.Login -200
POST http://localhost:8080/auth/login
Content-Type: application/json
{
"username": "prof",
"password": "1234"
}
###
4.WITHOUT token -403
spring-boot:build-image
spring-boot:build-image-no-fork
spring-boot:build-info
spring-boot:help
spring-boot:process-aot
spring-boot:process-test-aot
spring-boot:repackage
spring-boot:run
spring-boot:start
spring-boot:stop
spring-boot:test-run
surefire (org.apache.maven.plugins:maven-sure
Dependencies
Repositories
Services
HTTP Request
POST test | #2 Status: 200 (229 ms)
POST test | #3 Status: 200 (209 ms)
POST test | #5 Status: 403 (140 ms)
POST test | #6
GET test | 1. Public endpoint - 200 Status: 200 (107 ms)
Docker
HTTP/1.1 200
(Headers) …Content-Type: application/json…
{
"token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJwcm9mIiwiaWF0IjoxNzc4MzQ4MTQwLCJleHAiOjE3NzgOMz
}
Response file saved.
> 2026-05-09T193540.200.json
Response code: 200; Time: 209ms (209 ms); Content length: 141 bytes (141 B)